INSTITUT TEKNOLOGI DEL
MATERI PRAKTIKUM
4231101– Dasar Pemrograman
SEMESTER I TAHUN AJAR 2026/2027

Session Date	:	Sesi 2 dan 3 Tahun 2026
Semester	:	1.
Courses	:	Dasar Pemrograman
Week/Session	:	5/1.
Key Topics	:	Looping/Iteration Statement
Activity	:	Praktikum
Duration	:	110 menit
Delivery	:	e course
Deadline of delivery	:	**Akhir Sesi 3**
Place of delivery	:	e course
Goal	:	Mahasiswa memahami tentang materi loop dan nestedloop pada Bahasa C.

A. Pernyataan while

Sintaks: `while (kondisi) { statement; }`. Kondisi diperiksa sebelum body dijalankan. Selama kondisi bernilai true, body diulang. Jika kondisi selalu true, terjadi infinite loop. Pastikan ada pernyataan yang mengubah variabel kontrol di dalam body.

Praktik while Counter-Controlled: Jumlah 1 sampai N

Tujuan: memahami tiga unsur counter-controlled loop (inisialisasi, kondisi, update) pada `while`.

Kode Program

Simpan sebagai NIM_while_jumlahN.c

```c
#include <stdio.h>

int main(void) {
    int n, i = 1, jumlah = 0;

    printf("Masukkan N: ");
    scanf("%d", &n);

    while (i <= n) {
        jumlah += i;
        printf("i = %d, jumlah sementara = %d\n", i, jumlah);
        i++;
    }
    printf("Jumlah 1 sampai %d = %d\n", n, jumlah);
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Masukkan N: 5
i = 1, jumlah sementara = 1
i = 2, jumlah sementara = 3
i = 3, jumlah sementara = 6
i = 4, jumlah sementara = 10
i = 5, jumlah sementara = 15
Jumlah 1 sampai 5 = 15
```

Penjelasan Kode

- `int n, i = 1, jumlah = 0;` adalah inisialisasi. `i` adalah variabel kontrol, sedangkan `jumlah` adalah akumulator yang harus dimulai dari 0.
- `while (i <= n)` adalah kondisi pengulangan. Dicek sebelum setiap iterasi.
- `jumlah += i;` menambahkan nilai `i` ke total, sama dengan `jumlah = jumlah + i`.
- `i++;` adalah update. Tanpa baris ini `i` tidak berubah dan program menjadi infinite loop.

Tabel penelusuran (trace) untuk N = 5

Iterasi	i (awal)	i <= 5	jumlah (akhir)
1	1	true	1
2	2	true	3
3	3	true	6
4	4	true	10
5	5	true	15
Cek akhir	6	false (keluar)	15

Penjelasan Hasil

- Body dijalankan tepat 5 kali, sehingga muncul 5 baris "i = ..., jumlah sementara = ...".
- Kolom jumlah sementara naik 1, 3, 6, 10, 15. Setiap nilai adalah total dari 1 sampai i.
- Setelah `i` menjadi 6, kondisi `6 <= 5` bernilai false. Loop berhenti dan baris terakhir mencetak hasil akhir 15.
- Hasil dapat diverifikasi dengan rumus n(n+1)/2 = 5 x 6 / 2 = 15.

Praktik while Sentinel-Controlled: Rata-rata Skor

Tujuan: menggunakan nilai sentinel (-99) untuk mengakhiri input yang jumlahnya tidak diketahui.

Kode Program

Simpan sebagai NIM_while_sentinelSkor.c

```c
#include <stdio.h>
#define SENTINEL -99

int main() {
    int skor, jumlah = 0, banyak = 0;

    printf("Masukkan skor (%d untuk berhenti): ", SENTINEL);
    scanf("%d", &skor);                  /* baca data pertama */
    while (skor != SENTINEL) {
        jumlah += skor;
        banyak++;
        printf("Masukkan skor berikutnya (%d untuk berhenti): ", SENTINEL);
        scanf("%d", &skor);              /* baca data berikutnya */
    }

    if (banyak > 0)
        printf("Banyak data = %d, jumlah = %d, rata-rata = %.2f\n",
               banyak, jumlah, (double)jumlah / banyak);
    else
        printf("Tidak ada data yang dimasukkan.\n");
    return 0;
}
```

Hasil Eksekusi (dua kali percobaan)

Teks kuning tebal adalah masukan yang diketik pengguna.

Percobaan 1: tiga skor lalu sentinel

```
Masukkan skor (-99 untuk berhenti): 80
Masukkan skor berikutnya (-99 untuk berhenti): 90
Masukkan skor berikutnya (-99 untuk berhenti): 75
Masukkan skor berikutnya (-99 untuk berhenti): -99
Banyak data = 3, jumlah = 245, rata-rata = 81.67
```

Percobaan 2: langsung sentinel

```
Masukkan skor (-99 untuk berhenti): -99
Tidak ada data yang dimasukkan.
```

Penjelasan Kode

- `#define SENTINEL -99` mendefinisikan konstanta penanda akhir data. Nilai sentinel harus berada di luar rentang data yang valid.
- Pola dasar: baca data pertama sebelum loop, lalu baca data berikutnya di akhir body. Dengan pola ini sentinel tidak ikut dijumlahkan.
- `banyak++` menghitung jumlah data, sedangkan `jumlah += skor` menjumlahkan nilainya.
- `(double)jumlah / banyak` melakukan type casting. Tanpa cast, 245 / 3 adalah pembagian bilangan bulat dan hasilnya 81, bukan 81.67.
- `if (banyak > 0)` melindungi program dari pembagian dengan nol bila tidak ada data.

Penjelasan Hasil

- Percobaan 1: skor 80, 90, 75 diproses, lalu -99 menghentikan loop. Jumlah 245, banyak data 3, rata-rata 245 / 3 = 81.67.
- Percobaan 2: -99 diketik pertama kali, sehingga kondisi `skor != SENTINEL` langsung false. Body tidak dijalankan sama sekali dan program mencetak pesan tidak ada data.

- Ini membuktikan sifat `while`: dapat dijalankan 0 kali karena kondisi dicek di awal.

Praktik while Kondisi Umum: Manipulasi Digit

Tujuan: memakai `while` dengan kondisi umum (bukan hitungan atau sentinel) untuk memproses digit bilangan.

Kode Program

Simpan sebagai NIM_while_manipulasiDigit.c

```c
#include <stdio.h>

int main() {
    int n, asli, digit, banyak = 0, jumlah = 0, balik = 0;

    printf("Masukkan bilangan bulat positif: ");
    scanf("%d", &n);
    asli = n;

    while (n > 0) {
        digit = n % 10;            /* ambil digit paling kanan */
        banyak++;
        jumlah += digit;
        balik = balik * 10 + digit;
        printf("n = %4d | digit = %d | balik = %d\n", n, digit, balik);
        n = n / 10;                /* buang digit paling kanan */
    }
    printf("Bilangan %d: %d digit, jumlah digit = %d, dibalik = %d\n",
           asli, banyak, jumlah, balik);
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Masukkan bilangan bulat positif: 4821
n = 4821 | digit = 1 | balik = 1
n =  482 | digit = 2 | balik = 12
n =   48 | digit = 8 | balik = 128
n =    4 | digit = 4 | balik = 1284
Bilangan 4821: 4 digit, jumlah digit = 15, dibalik = 1284
```

Penjelasan Kode

- `n % 10` menghasilkan digit paling kanan, sedangkan `n / 10` (pembagian bilangan bulat) membuang digit itu.
- `balik = balik * 10 + digit` menggeser hasil satu posisi ke kiri lalu menambahkan digit baru, sehingga digit tersusun terbalik.
- Nilai asli disimpan di `asli` karena `n` akan habis (menjadi 0) di dalam loop.
- Kondisi `n > 0` membuat jumlah iterasi bergantung pada banyaknya digit, bukan angka tetap.

Tabel penelusuran untuk n = 4821

Iterasi	n	digit = n % 10	balik	n / 10
1	4821	1	1	482
2	482	2	12	48
3	48	8	128	4

Iterasi	n	digit = n % 10	balik	n / 10
4	4	4	1284	0 (berhenti)

Penjelasan Hasil

- Loop berjalan 4 kali karena 4821 memiliki 4 digit. Setiap baris keluaran sama dengan satu baris tabel penelusuran.
- Jumlah digit 1 + 2 + 8 + 4 = 15, dan bilangan terbalik 1284.
- Jika pengguna memasukkan 0 atau bilangan negatif, kondisi `n > 0` langsung false. Hasilnya 0 digit, dan program perlu ditambah validasi untuk kasus ini (lihat latihan).

Latihan Mandiri: while

1. Cetak bilangan kelipatan 3 dari 3 sampai 30 memakai `while`.
2. Ubah W-2 agar juga menampilkan skor tertinggi dan terendah.
3. Perbaiki W-3 agar menolak masukan 0 atau negatif dengan pesan kesalahan.

B. Pernyataan do-while

Sintaks: `do { statement; } while (kondisi);`. Body dijalankan dahulu, baru kondisi dicek. Karena itu body dijalankan minimal satu kali. Jangan lupa tanda `;` setelah `while (kondisi)`.

Praktik do-while: Validasi Nilai 0-100

Tujuan: menggunakan `do-while` sebagai input validation loop.

Kode Program

Simpan sebagai NIM_do-while_validasiNilai.c

```c
#include <stdio.h>

int main() {
    int nilai;

    do {
        printf("Masukkan nilai (0-100): ");
        scanf("%d", &nilai);
        if (nilai < 0 || nilai > 100)
            printf("Nilai tidak valid, silakan ulangi.\n");
    } while (nilai < 0 || nilai > 100);

    printf("Nilai diterima: %d\n", nilai);
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Masukkan nilai (0-100): 150
Nilai tidak valid, silakan ulangi.
Masukkan nilai (0-100): -5
Nilai tidak valid, silakan ulangi.
Masukkan nilai (0-100): 85
Nilai diterima: 85
```

Penjelasan Kode

- Blok `do { ... }` dijalankan dahulu, baru kondisi `while (...)` diperiksa. Karena itu prompt pasti tampil minimal satu kali.
- Kondisi `nilai < 0 || nilai > 100` bernilai true bila nilai tidak valid. Operator `||` berarti salah satu syarat cukup.
- Pesan kesalahan dicetak di dalam body menggunakan kondisi yang sama dengan kondisi loop.
- Tanda `;` setelah `while (...)` wajib pada `do-while`.

Penjelasan Hasil

- Masukan 150 melanggar batas atas, sehingga pesan kesalahan muncul dan loop berulang.
- Masukan -5 melanggar batas bawah, sehingga pesan kesalahan muncul lagi dan loop berulang.
- Masukan 85 berada pada rentang 0-100, kondisi menjadi false, dan loop berhenti. Nilai 85 diterima.
- Total prompt muncul 3 kali, sesuai banyaknya percobaan pengguna.

Catatan: Variabel `nilai` tidak perlu diberi nilai awal karena body dijalankan lebih dulu sebelum kondisi dicek. Inilah keunggulan `do-while` untuk input.

Praktik do-while: Meminta Bilangan Genap

Tujuan: memahami bahwa `do-while` mengulang selama kondisi true, dan menghitung jumlah percobaan.

Kode Program

Simpan sebagai NIM_do-while_bilGenap.c

```c
#include <stdio.h>

int main() {
    int num, percobaan = 0;

    do {
        printf("Masukkan bilangan genap: ");
        scanf("%d", &num);
        percobaan++;
    } while (num % 2 != 0);

    printf("Bilangan genap %d diterima setelah %d percobaan.\n", num,
    percobaan);
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Masukkan bilangan genap: 7
Masukkan bilangan genap: 3
Masukkan bilangan genap: 8
Bilangan genap 8 diterima setelah 3 percobaan.
```

Penjelasan Kode

- Kondisi `num % 2 != 0` bernilai true untuk bilangan ganjil (sisa bagi 2 tidak nol). Loop berulang selama pengguna memasukkan ganjil.
- `percobaan++` ditempatkan di dalam body agar setiap input dihitung, termasuk input terakhir yang benar.
- Setelah loop, `num` berisi bilangan genap yang valid.

Tabel penelusuran

Percobaan	Input	num % 2	Kondisi (!= 0)	Aksi
1	7	1	true	ulangi
2	3	1	true	ulangi
3	8	0	false	berhenti

Penjelasan Hasil

- Dua masukan pertama (7 dan 3) ganjil sehingga loop mengulang tanpa pesan.
- Masukan 8 genap, kondisi false, dan loop selesai.
- Keluaran akhir "diterima setelah 3 percobaan" cocok dengan jumlah baris prompt pada layar.

Praktik do-while: Menu Berulang

Tujuan: membuat program menu yang terus tampil sampai pengguna memilih keluar.

Kode Program

Simpan sebagai NIM_do-while_menuBerulang.c

```c
#include <stdio.h>

int main() {
    int pilihan;
    double a, b;

    do {
        printf("\n=== MENU KALKULATOR ===\n");
        printf("1. Tambah\n2. Kurang\n3. Keluar\nPilihan: ");
        scanf("%d", &pilihan);

        if (pilihan == 1 || pilihan == 2) {
            printf("Masukkan dua bilangan: ");
            scanf("%lf %lf", &a, &b);
            if (pilihan == 1)
                printf("Hasil: %.2f\n", a + b);
            else
                printf("Hasil: %.2f\n", a - b);
        } else if (pilihan != 3) {
            printf("Pilihan tidak dikenal.\n");
        }
    } while (pilihan != 3);

    printf("Program selesai.\n");
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
=== MENU KALKULATOR ===
1. Tambah
2. Kurang
3. Keluar
Pilihan: 1
Masukkan dua bilangan: 5 3
Hasil: 8.00

=== MENU KALKULATOR ===
1. Tambah
2. Kurang
3. Keluar
Pilihan: 2
Masukkan dua bilangan: 10 4
Hasil: 6.00

=== MENU KALKULATOR ===
1. Tambah
2. Kurang
3. Keluar
Pilihan: 9
Pilihan tidak dikenal.

=== MENU KALKULATOR ===
1. Tambah
2. Kurang
3. Keluar
Pilihan: 3
Program selesai.
```

Penjelasan Kode

- Menu harus tampil minimal sekali, sehingga `do-while` lebih tepat daripada `while`.
- `if (pilihan == 1 || pilihan == 2)` menangani dua operasi. Di dalamnya ada `if` bersarang untuk memilih tambah atau kurang (conditional di dalam loop).
- `else if (pilihan != 3)` menangkap pilihan tak dikenal. Pilihan 3 sengaja tidak dicetak sebagai kesalahan.
- `while (pilihan != 3)` adalah kondisi lanjut. Angka 3 berfungsi sebagai sentinel.
- `scanf("%lf %lf", &a, &b)` membaca dua bilangan desimal dalam satu baris, dipisahkan spasi.

Penjelasan Hasil

- Pilihan 1 dengan 5 dan 3 menghasilkan 8.00 (penjumlahan).
- Pilihan 2 dengan 10 dan 4 menghasilkan 6.00 (pengurangan).
- Pilihan 9 tidak dikenal, sehingga muncul pesan dan menu ditampilkan lagi.
- Pilihan 3 membuat kondisi false. Loop berhenti dan "Program selesai." tercetak.
- Menu muncul 4 kali, sesuai 4 pilihan yang dimasukkan.

Latihan Mandiri: do-while

1. Minta pengguna memasukkan PIN 4 digit (1000-9999) sampai valid.
2. Tambahkan operasi kali dan bagi pada D-3, serta tolak pembagian dengan nol.

3. Buat permainan tebak angka: program menyimpan angka rahasia 42, lalu memberi petunjuk "terlalu besar" atau "terlalu kecil" sampai tebakan benar.

C. Pernyataan for

Sintaks: `for (inisialisasi; kondisi; update) statement;`. Ketiga unsur kontrol berada dalam satu baris, sehingga `for` ideal untuk counting loop. Urutan eksekusinya: inisialisasi (sekali), cek kondisi, body, update, lalu kembali ke cek kondisi.

Praktik for: Faktorial

Tujuan: memakai `for` untuk counting loop dengan jumlah ulangan yang diketahui.

Kode Program

Simpan sebagai NIM_forFaktorial.c

```c
#include <stdio.h>

int main() {
    int n, i;
    long long fakt = 1;

    printf("Masukkan n: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        fakt *= i;
        printf("%d! = %lld\n", i, fakt);
    }
    printf("Jadi %d! = %lld\n", n, fakt);
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Masukkan n: 6
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
Jadi 6! = 720
```

Penjelasan Kode

- Sintaks `for (inisialisasi; kondisi; update)`: `i = 1` dijalankan sekali, `i <= n` dicek tiap iterasi, `i++` dijalankan di akhir tiap iterasi.
- `fakt` dimulai dari 1, bukan 0, karena 1 adalah elemen identitas perkalian. Jika dimulai 0, semua hasil menjadi 0.
- `long long` dan `%lld` dipakai karena faktorial tumbuh sangat cepat. 13! sudah melebihi batas `int` (sekitar 2,1 miliar), dan 20! masih muat di `long long`.
- Mencetak di dalam loop memperlihatkan hasil kumulatif setiap tahap.

Penjelasan Hasil

- Loop berjalan 6 kali untuk n = 6, dan setiap baris adalah hasil kali sebelumnya dikalikan `i`.

- Nilainya 1, 2, 6, 24, 120, 720. Contohnya 5! = 24 x 5 = 120 dan 6! = 120 x 6 = 720.
- Baris terakhir mengulang hasil akhir, yaitu 6! = 720.

Praktik for: Tabel Perkalian

Tujuan: memakai `for` dengan batas tetap dan memformat keluaran agar rapi.

Kode Program

Simpan sebagai NIM_forPerkalian.c

```c
#include <stdio.h>

int main() {
    int n, i;

    printf("Tabel perkalian berapa? ");
    scanf("%d", &n);

    for (i = 1; i <= 10; i++) {
        printf("%d x %2d = %3d\n", n, i, n * i);
    }
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Tabel perkalian berapa? 7
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
7 x  4 =  28
7 x  5 =  35
7 x  6 =  42
7 x  7 =  49
7 x  8 =  56
7 x  9 =  63
7 x 10 =  70
```

Penjelasan Kode

- Loop `for (i = 1; i <= 10; i++)` berjalan tepat 10 kali. Perhatikan `<=` agar angka 10 ikut diproses.
- Format `%2d` dan `%3d` memberi lebar minimal sehingga angka satu digit dan dua digit tetap sejajar.
- `n * i` dihitung di dalam `printf` tanpa variabel tambahan.

Penjelasan Hasil

- Untuk n = 7, setiap baris adalah 7 dikali i, dari 7 x 1 = 7 sampai 7 x 10 = 70.
- Hasil naik tetap 7 setiap baris (7, 14, 21, ...), sesuai sifat deret aritmetika.
- Kolom rapi karena lebar format tetap, misalnya " 1" dan "10" sama-sama dua karakter.

Praktik for: Penggajian Karyawan

Tujuan: menerapkan contoh `for` dari slide (payroll) dengan akumulator total.

Kode Program

Simpan sebagai NIM_forPenggajian.c

```c
#include <stdio.h>

int main() {
    int count_emp, number_emp = 3;
    double hours, rate, pay, total_pay = 0.0;

    for (count_emp = 0; count_emp < number_emp; count_emp++) {
        printf("Karyawan ke-%d\n", count_emp + 1);
        printf("  Jam kerja      > ");
        scanf("%lf", &hours);
        printf("  Tarif per jam  > ");
        scanf("%lf", &rate);
        pay = hours * rate;
        printf("  Gaji           = Rp%.2f\n\n", pay);
        total_pay = total_pay + pay;
    }
    printf("Semua karyawan selesai diproses\n");
    printf("Total gaji = Rp%.2f\n", total_pay);
    return 0;
}
```

Hasil Eksekusi

Teks kuning tebal adalah masukan yang diketik pengguna.

```
Karyawan ke-1
  Jam kerja      > 40
  Tarif per jam  > 15000
  Gaji           = Rp600000.00

Karyawan ke-2
  Jam kerja      > 35
  Tarif per jam  > 20000
  Gaji           = Rp700000.00

Karyawan ke-3
  Jam kerja      > 20
  Tarif per jam  > 25000
  Gaji           = Rp500000.00

Semua karyawan selesai diproses
Total gaji = Rp1800000.00
```

Penjelasan Kode

- `count_emp = 0` adalah inisialisasi, `count_emp < number_emp` adalah kondisi, dan `count_emp++` adalah update. Karena mulai dari 0, label karyawan memakai `count_emp + 1`.
- `scanf("%lf", ...)` dipakai untuk membaca `double`, sedangkan `printf` memakai `%f` (di sini `%.2f` untuk dua desimal).
- `total_pay` diawali 0.0 dan diakumulasi: `total_pay = total_pay + pay`.
- Pesan "Semua karyawan selesai diproses" dan total berada di luar loop, sehingga hanya tercetak sekali.

Penjelasan Hasil

- Karyawan 1: 40 x 15000 = 600000. Karyawan 2: 35 x 20000 = 700000. Karyawan 3: 20 x 25000 = 500000.

- Total 600000 + 700000 + 500000 = 1800000, sama dengan keluaran akhir.
- Blok input dan hitung berulang 3 kali karena `number_emp` bernilai 3, lalu kontrol keluar dari loop.

Latihan Mandiri: for

1. Cetak deret Fibonacci 10 suku pertama.
2. Hitung jumlah dan rata-rata N bilangan yang dimasukkan pengguna.
3. Cetak bilangan 10 sampai 1 (hitung mundur) dengan decrement `i--`.

Submission:

1. Kumpulkan semua code yang telah anda buat selama sesi 2 dan 3 ini ke ecourse.
2. Sesuaikan nama file yang anda buat dengan apa yang sudah ditentukan.
3. Kumpulkan dalam bentuk .zip. dengan format file NIM_Tugas5_Looping.zip

   Contoh: 42326001_Tugas5_Looping.zip

online-pdf-no-copy.com